

UMGT ECU — Firmware Upload Guide

How to build and upload the ESP32 firmware with PlatformIO

The ECU's brain is an ESP32 board. Everything it does — reading thermocouples, driving pumps, talking to the ESC and Pixhawk — is one program (the *firmware*) held in a single source file. To change how the ECU behaves you edit that file, **build** it (compile it on the PC), then **upload** it to the board over USB. This guide covers exactly that, start to finish. No coding knowledge required — just the steps.

You will need

- The ECU (the ESP32 board on the shield)
- A **USB-C data cable** (not a charge-only cable)
- A Windows PC with PlatformIO installed (see Section 2)

1 Connect the ECU to the computer

The ESP32 board has a **USB-C** connector on its short edge. Plug USB-C into the ESP32 and the other end into the PC. Power the shield as usual; the USB link is only for programming and the serial console.

- Use a **data** cable. Many USB cables carry power only — the board will light up but never appear on the PC. If nothing shows up, suspect the cable first.
- The board uses a **CH340** USB-to-serial chip. Windows assigns it a COM port number, e.g. **COM4** or **COM5**. You'll need that number.
- If no COM port ever appears, install the **CH340 driver** (search “CH340 driver Windows”), unplug and replug.

Close the dashboard first

A COM port can only be owned by **one** program at a time. If the UMGT Engine dashboard (or any serial monitor) is **Connected** to the board, the upload will fail with “port is busy”. Click **Disconnect** in the dashboard before uploading — you do not have to close the whole app.

2 Set up PlatformIO (one-time)

PlatformIO is the free tool that compiles the code and flashes it to the board. There are two ways to use it — pick one.

Option A — Visual Studio Code (recommended, has buttons)

- 1 Install **Visual Studio Code** (free, from Microsoft).
- 2 Open VS Code → click the **Extensions** icon on the left (four squares) → search **PlatformIO IDE** → **Install**.
- 3 Let it finish downloading its toolchain, then restart VS Code when it asks. A small **PlatformIO** (alien head) icon and a blue bottom toolbar appear when it's ready.

Option B — Command line (already installed on this PC)

PlatformIO Core is already installed here, so you can skip Option A if you prefer typing commands. The program is:

```
C:\Users\owen.maute\.platformio\penv\Scripts\pio.exe
```

If you type `pio` in a terminal and it says “not recognized”, use that full path in place of `pio` in every command below. To install PlatformIO Core on another PC: `pip install platformio`.

3 Where the files live

Everything for the ECU is in the project repo. Two files matter for uploading:

Item	Location / value
Repo root	C:\Users\owen.maute\UMGTECU2026
PlatformIO project	...\UMGTECU2026\UMGT
Firmware source (edit this)	...\UMGT\src\main.ino.cpp
Build config	...\UMGT\platformio.ini

Open the right folder

In VS Code use **File** → **Open Folder** and open the **UMGT** folder — the one that directly contains `platformio.ini`. If you open the repo root or the wrong level, PlatformIO won't recognise the project and the Build/Upload buttons won't appear.

main.ino.cpp is the entire ECU program — the pin map, the thermocouple/ESC/Pixhawk code, the start sequence, all of it. It is the *only* file you edit to change behaviour.

platformio.ini tells PlatformIO which board and port to use. It currently reads:

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200
upload_speed = 115200
monitor_port = COM4
upload_port = COM4
```

The only line you may need to change is **upload_port** (and `monitor_port`) — see Section 6 on picking the right COM number.

4 Verify (build/compile) the code

Always build before you upload. Building compiles the code on the PC and catches mistakes there — a typo caught on the PC is harmless; the same typo flashed to the board could leave the ECU dead. Building does *not* touch the board, so it's completely safe to do anytime.

In VS Code:

- Click the **checkmark** (✓) in the blue bottom toolbar, **or**
- open the PlatformIO sidebar → **esp32dev** → **General** → **Build**.

On the command line (from inside the UMGT folder):

```
cd C:\Users\owen.maute\UMGTECU2026\UMGT
pio run -e esp32dev
```

A good build ends with a green **SUCCESS** and a memory summary like:

```
RAM: [= ] 7.2% (used 23592 bytes ...)
Flash: [=== ] 28.7% (used 376529 bytes ...)
===== [SUCCESS] Took 8.0 seconds =====
```

If instead you get **errors**, PlatformIO prints each one with the file and line number (e.g. `main.ino.cpp:1234: error: ...`). Nothing was sent to the board. Fix the line, build again, repeat until it says **SUCCESS**.

5 Upload to the ECU

Once the build is **SUCCESS** and the dashboard is disconnected, flash it:

In VS Code:

- Click the **right-arrow** (→) in the blue bottom toolbar, **or**
- PlatformIO sidebar → **esp32dev** → **General** → **Upload**.

On the command line:

```
pio run -e esp32dev -t upload
```

If the board is on a different COM port than `platformio.ini` lists, name it explicitly (nothing else needs to change):

```
pio run -e esp32dev -t upload --upload-port COM5
```

A successful upload scrolls through these stages:

- Connecting.....
- Writing at 0x00010000... (12%) ... climbing to 100%
- Hash of data verified.
- Hard resetting via RTS pin...

Hash of data verified plus **Hard resetting** means it worked — the ECU reboots straight into the new firmware. You can reconnect the dashboard now.

6 Picking the right COM port

This is the most common gotcha

The ESP32 does not always keep the same COM number. It may come up as **COM4** one time and **COM5** the next — especially after unplugging and replugging. `platformio.ini` is set to `COM4`, but the board is often on `COM5`. When in doubt, check.

Find the current port two ways:

- **Device Manager** → **Ports (COM & LPT)** → look for **USB-SERIAL CH340 (COMx)**. That COMx is your board.

- Or list it from PlatformIO: `pio device list`.

Then either edit the two `...port = COMx` lines in `platformio.ini`, or just pass `--upload-port COMx` on the command line for a one-off. (COM1 is *not* your board — that's a built-in port; ignore it.)

7 Troubleshooting

Symptom	Fix
"Could not open COMx, the port is busy"	The dashboard or a serial monitor owns the port. Disconnect it, then upload.
"Failed to connect" / "Timed out waiting for packet header"	Hold the BOOT button on the ESP32 while it says <i>Connecting.....</i> , release once <i>Writing</i> starts. Also try a different USB port or cable.
No COM port appears at all	Charge-only cable, or the CH340 driver isn't installed. Swap to a data cable / install the driver, then replug.
Wrong port / uploads to nothing	Board re-enumerated to a different COM number — see Section 6, pass <code>--upload-port</code> .
Build fails with red errors	A code mistake — nothing was flashed. Fix the file:line it names and build again.
Uploads OK but the ECU misbehaves	Confirm you actually edited <code>UMGT\src\main.ino.cpp</code> and that the build was SUCCESS before uploading.
Board keeps dropping off USB mid-session	Usually a power/grounding issue on the rig rather than software — reset power and USB, avoid unpowered USB hubs.

8 Quick reference

Command-line cheat sheet (run these from inside the UMGTECU folder):

Do this	Command
Go to the project	<code>cd C:\Users\owen.maute\UMGTECU2026\UMGT</code>
Build / verify	<code>pio run -e esp32dev</code>
Upload	<code>pio run -e esp32dev -t upload</code>
Upload to a specific port	<code>pio run -e esp32dev -t upload --upload-port COM5</code>
List connected boards	<code>pio device list</code>
Open the serial console	<code>pio device monitor -b 115200</code>
Clean build files	<code>pio run -e esp32dev -t clean</code>

Golden rule: **Disconnect the dashboard** → **Build (✓)** → **Upload (→)** → **wait for "Hash of data verified"**.

UMGT ECU project — firmware upload guide. Source of truth: `UMGT\src\main.ino.cpp` and `UMGT\platformio.ini`.